Clean Code - Chapter 3: Functions (Full Notes)

1. Small Functions

Functions should be small (ideally <20 lines). Smaller functions are easier to read, test, and maintain.

```
// BAD
public void processEverything() {
    // 100+ lines
}

// GOOD
public void process() {
    validate();
    calculate();
    save();
}
```

2. Do One Thing

A function should do ONE thing, do it well, and do it only.

```
// BAD
public void handleUser() {
    validateUser();
    saveUser();
    sendEmail();
}

// GOOD
public void saveUser(User user) {
    validate(user);
    persist(user);
}
```

3. One Level of Abstraction

Do not mix high-level logic with low-level details.

```
// BAD
public void process() {
    User user = getUser();
    String name = user.getName();
    db.save(name);
}

// GOOD
public void process() {
    User user = getUser();
    saveUser(user);
}
```

4. Function Arguments (IMPORTANT)

0 Arguments (BEST): easiest to understand & test.

```
public void save() {}
```

1 Argument (GOOD): common and clear.

```
public User getUser(String id) {}
```

2 Arguments (OK but careful): ordering matters.

```
public void updateUser(String id, String name) {}
```

3+ Arguments (BAD): use object.

```
// BAD
createUser(name, age, address, phone);

// GOOD
createUser(User user);
```

Avoid flag arguments:

```
// BAD
render(true);

// GOOD
renderAdmin();
renderUser();
```

5. Use Descriptive Names

Use long, meaningful names instead of short unclear ones.

```
// BAD
public void f() {}

// GOOD
public void calculateMonthlySalary() {}
```

6. Avoid Side Effects

Functions should not hide extra behavior.

```
// BAD
public boolean checkPassword() {
    Session.initialize(); // hidden
    return true;
}
```

7. Command Query Separation

Function should either DO or RETURN, not both.

```
// BAD
if(set("username", "john")) {}
```

```
// GOOD
setUsername("john");
if(usernameExists()) {}
```

8. Error Handling

Use exceptions instead of error codes.

```
// BAD
if(delete() == ERROR) {}

// GOOD
try {
    delete();
} catch(Exception e) {
    log(e);
}
```

9. DRY (Don't Repeat Yourself)

Avoid duplication to reduce bugs.

```
// BAD
if(type == A) doA();
if(type == B) doB();

// GOOD
handle(type);
```

10. How to Write Clean Functions

Start messy → refactor → split → rename → remove duplication → keep tests passing.

Conclusion

Functions are the verbs of a system. Clean functions are small, focused, well-named, and readable.